Viet Nam National University, HCMC

University of Science

Faculty of Information Technology

Assignment 01: Multi-Label Digit Classification Using CNNs

CS414 | Machine Learning

Student:
Tran Nguyen Phuc Thinh (23125072)
Doan Cong Pho (23125089)
Nguyen Duc Thinh (23125092)

Lecturer:
Nguyen Vinh Tiep
Teaching Assistant:
Nguyen Trong Thuan
Ho Le Minh Quan

July 29, 2026



Contents

1 Dataset Description **2**
1.1 Multi-label MNIST . . . 2
1.1.1 Image Statistics . . . 2
1.1.2 Label Balance . . . 3
1.1.3 Digit Presence . . . 3
1.1.4 Number of Digits per Image . . . 3
1.1.5 Actual Digit Occurrences . . . 4
1.2 MNIST . . . 4
1.3 Generated MNIST . . . 5

2 Experiments **7**
2.1 Baseline . . . 7
2.2 ResNet-50 . . . 7
2.3 ResNet-18 . . . 8
2.4 ResNet-18 + Regularization . . . 9
2.4.1 Parameters Tuning and Learning Rate Scheduling . . . 9
2.4.2 Histogram Prediction . . . 9
2.4.3 Contrastive Learning and Embedding Loss . . . 10
2.4.4 Drop Path . . . 11
2.4.5 Data Generation . . . 11

3 Results **13**

References **14**

1 Dataset Description

1.1 Multi-label MNIST

The provided Multi-label MNIST dataset consists of 64×64 grayscale images containing multiple MNIST digits. For our architecture, the original images are replicated across three channels, producing tensors of shape $64 \times 64 \times 3$. Pixel values are normalized to the range $[0, 1]$.

Each image is associated with a 10-dimensional multi-hot label vector. A value of one indicates that the corresponding digit appears at least once in the image. The dataset also provides metadata including the digit-count histogram, the total number of digits, the label of the central digit, the digit label for each slot, and the top-left coordinates of each digit's 28×28 bounding box of the original MNIST dataset.

Table 1: Multi-label MNIST dataset splits.

Split	**Images**	**Image Shape**	**Label Shape**
Train	50,000	$64 \times 64 \times 3$	$50,000 \times 10$
Validation	10,000	$64 \times 64 \times 3$	$10,000 \times 10$
Test	10,000	$64 \times 64 \times 3$	$10,000 \times 10$

The metadata associated with each sample contains the following fields:

- `center_labels`: the digit located at the central position;
- `count_labels`: a 10-dimensional histogram of digit occurrences;
- `num_digits`: the total number of digit instances;
- `all_digit_labels`: the digit assigned to each slot, padded with -1;
- `all_positions`: the top-left coordinates of each 28×28 digit bounding box;
- `label_type`: the type of classification task.

1.1.1 Image Statistics

The image statistics are similar across all three splits, indicating that there is no significant distribution shift between training, validation, and test data.

Table 2: Image statistics (normalized) for the Multi-label MNIST dataset.

Split	**Mean**	**Std.**	**Min**	**Max**
Train	0.1864	0.3025	0.0000	1.0000
Validation	0.1853	0.3017	0.0000	1.0000
Test	0.1885	0.3049	0.0000	1.0000

1.1.2 Label Balance

The dataset is relatively balanced at the label level. Approximately 52% of the ten possible digit labels are positive for each image.

Table 3: Positive and negative multi-label statistics.

Split	Positive	Negative	Positive Ratio	Mean Positive Labels
Train	259,727	240,273	0.5195	5.19
Validation	51,891	48,109	0.5189	5.19
Test	52,018	47,982	0.5202	5.20

1.1.3 Digit Presence

Table 4 reports the number and percentage of images in which each digit appears at least once. Digit 1 is the most common class, while digit 5 is the least common. Nevertheless, the differences are relatively smalll so class imbalance will not be a concern.

Table 4: Digit presence frequencies.

Digit	Train	Validation	Test
0	25,651 (51.30%)	5,151 (51.51%)	5,231 (52.31%)
1	27,938 (55.88%)	5,825 (58.25%)	5,692 (56.92%)
2	26,279 (52.56%)	4,998 (49.98%)	5,266 (52.66%)
3	26,678 (53.36%)	5,175 (51.75%)	5,206 (52.06%)
4	25,214 (50.43%)	5,252 (52.52%)	5,203 (52.03%)
5	24,072 (48.14%)	4,873 (48.73%)	4,780 (47.80%)
6	25,763 (51.53%)	4,944 (49.44%)	5,026 (50.26%)
7	26,867 (53.73%)	5,338 (53.38%)	5,326 (53.26%)
8	25,530 (51.06%)	5,054 (50.54%)	5,122 (51.22%)
9	25,735 (51.47%)	5,281 (52.81%)	5,166 (51.66%)

1.1.4 Number of Digits per Image

The number of digit instances in each image ranges from two to eight. Images containing five or six digits are the most common.

It is important to distinguish between the number of digit instances and the number of positive multi-label classes. For example, an image containing the digits

$$[6, 8, 8, 2, 8, 8]$$

has six digit instances but only three positive multi-label classes: $\{2, 6, 8\}$. Repeated digits are counted multiple times in `count_labels`, but only once in the multi-hot label.

Table 5: Distribution of the number of digit instances per image.

Number of Digits	Train	Validation	Test
2	66	12	12
3	1,574	330	294
4	9,878	2,010	1,978
5	19,989	3,925	3,975
6	14,325	2,898	2,910
7	3,878	776	783
8	290	49	48

1.1.5 Actual Digit Occurrences

The training-set digit histogram is shown in Table 6. These values count repeated digits separately.

Table 6: Actual digit occurrences across the dataset splits.

Digit	Train	Validation	Test
0	34,260	6,987	6,936
1	39,037	8,318	8,030
2	35,411	6,647	7,133
3	36,347	6,962	7,116
4	33,511	7,046	6,942
5	31,461	6,450	6,200
6	34,749	6,474	6,667
7	36,443	7,258	7,256
8	34,264	6,713	6,794
9	34,544	7,086	6,984

The additional metadata is useful for auxiliary learning tasks. In particular, the digit-count histograms were used for histogram prediction and for constructing semantic similarity targets for the embedding branch, which we will discuss in later sections.

In conclusion, since the digit classes are relatively balanced across the training, validation, and test sets, no additional class-balancing technique was required. The positive-to-negative label ratio is also close to 1:1, so methods such as focal loss, class weighting, or oversampling were not necessary. The experiments therefore used standard losses without introducing extra mechanisms to compensate for severe class imbalance. The code for inspecting data is provided in `statistics.py`

1.2 MNIST

The provided Multi-label MNIST datasete is based on the MNIST dataset [1], which is a standard benchmark dataset for handwritten digit classification. It contains grayscale images of

digits from 0 to 9. Each image has a spatial resolution of 28×28 pixels, and each sample is associated with a single integer label in the range $[0, 9]$. The official dataset contains 60,000 training images and 10,000 test images. The dataset is available through the torchvision interface, which provides separate training and test splits using the `train` argument.

Table 7: Original MNIST dataset.

Property	**Value**
Training images	60,000
Test images	10,000
Number of classes	10
Classes	Digits 0–9
Image resolution	28×28
Image type	Grayscale
Label type	Single integer label

1.3 Generated MNIST

During experimentation, we saw that the model showed signs of overfitting, with the training exact-match accuracy becoming noticeably higher than the validation accuracy. To reduce this gap, an additional training set was generated using the original MNIST digit images. The generated images were used only for training, while the validation and test sets remained unchanged. This prevents the evaluation protocol from being affected by synthetic data.

Table 8: Configuration of the generated Multi-label MNIST dataset.

Property	**Value**
Number of generated images	100,000
Canvas size	64×64
Digit size	28×28
Source digit dataset	MNIST training set
Position metadata	`train.pt`
Digit sampling	Uniform over digits 0–9
Pixel composition	Transparent foreground pasting

The generation process reuses only the spatial metadata from `train.pt`. In particular, the original images and labels are not reused. For each generated sample, one position template is randomly selected from the original training metadata. The template determines the number of digit slots and the top-left coordinate of each 28×28 bounding box of the original MNIST dataset image.

A digit class is then sampled independently and uniformly from $\{0, \ldots, 9\}$ for every valid slot. Therefore, every digit class has a probability of 10% at each slot, preventing the generated data from developing a strong class preference. The position templates are sampled from the original training set, which preserves the distribution of digit counts and spatial layouts.

To avoid repeatedly using the same MNIST digit before the available examples of that class are exhausted, a shuffled queue is maintained separately for each digit class. When a queue becomes empty, it is reshuffled and reused. This provides deterministic but diverse sampling under a fixed random seed.

Each selected MNIST digit is pasted onto a blank 64×64 canvas at its assigned position. Only non-zero pixels overwrite the canvas:

$$\text{mask} = \text{digit} > 0,$$

$$\text{canvas_patch}[\text{mask}] = \text{digit}[\text{mask}].$$

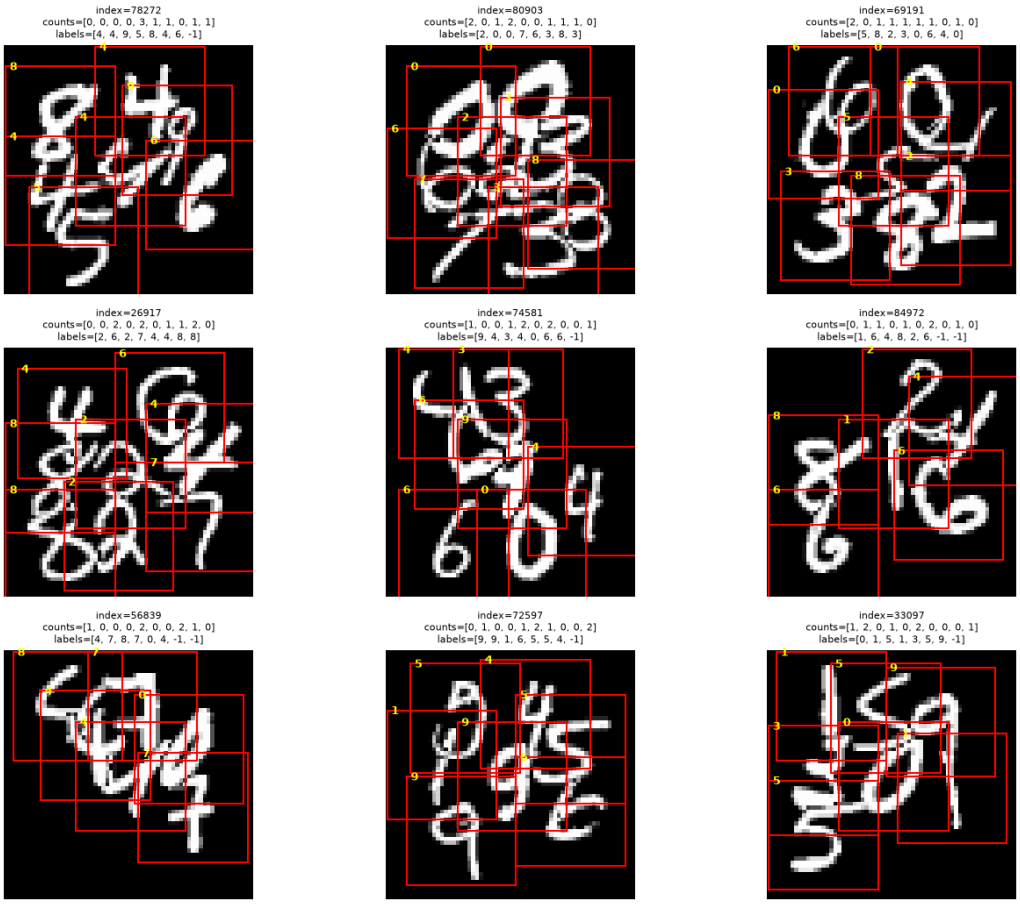


Figure 1: Generated samples with visualized bounding boxes.

Thus, black pixels remain transparent and no blending or maximum operation is performed. The labels are generated directly from the sampled digit classes. For each image, the pipeline stores a multi-hot label vector, a digit-count histogram, the total number of digit instances, the digit label of each slot, and the corresponding bounding-box coordinates.

The generated dataset is finally concatenated with the original training set during training. This increases the number of available training examples while retaining the original validation and test distributions for a fair comparison. The generation code is provided in **`generate_train_dataset.ipynb`**.

2 Experiments

2.1 Baseline

The baseline model is a small convolutional neural network implemented in `cnn.ipynb`. It receives a 24×24 grayscale image and consists of three convolutional blocks with 32, 64, and 128 filters, respectively. Each convolutional layer uses a 3×3 kernel with ReLU activation and is followed by max pooling. The resulting feature map is flattened and passed through a fully connected layer with 256 units and dropout before producing ten sigmoid outputs. Sigmoid activation is used because multiple digit classes may be present in one image.

The model was trained with binary cross-entropy and the Adam optimizer using a learning rate of 10^{-3}. The batch size was 128 and the maximum number of epochs was 100. Early stopping and ReduceLROnPlateau both monitored validation loss, with a patience of five epochs.

The lowest validation loss was reached at epoch 8, with a value of 0.3720. The model weights from this epoch were restored before evaluating the test set. The corresponding results are shown in Table 9.

Table 9: Baseline CNN performance.

Split	Exact Match	Binary Accuracy	Precision	Recall
Train	0.2458	0.8674	0.8841	0.8570
Test	0.1838	0.8394	0.8701	0.8127

Although the training metrics continued to improve after epoch 8, the training was stopped at epoch 13 because the validation loss did not improve for five consecutive epochs. This indicates that validation loss was not fully aligned with the main evaluation metric, exact-match accuracy. The baseline therefore provides a useful starting point, but its limited architecture and reactive learning-rate schedule restrict its performance. The following experiments focus on stronger architectures and improved regularization and scheduling strategies.

2.2 ResNet-50

ResNet-50 [2] was selected as a strong first experiment because it is a well-established convolutional architecture with a reliable residual learning design. Its skip connections make it easier to optimize than a plain deep network and help preserve useful low-level features while learning more complex representations in deeper layers. This is suitable for Multi-label MNIST, where the model must detect several digits that may appear at different locations in the same image.

The Keras ResNet-50 backbone was initialized randomly and used without its original classification head. A global average pooling layer was followed by a fully connected layer and dropout. The resulting representation was used for two prediction tasks: multi-label digit classification and digit-count histogram prediction. The main evaluation metric was exact-match accuracy.

The model was trained using the training split, while the validation split was used for model selection. The checkpoint with the lowest val loss was selected for final evaluation on the test set. Here, we display the metrics of evaluation of the model on the val set and metrics of that same model during training.

Table 10: ResNet-50 performance on the train and test sets.

Split	Exact Match	Binary Accuracy	Precision	Recall
Train	0.9210	0.9899	0.9933	0.9872
Test	0.3985	0.9045	0.9227	0.8911

The train values in Table 10 correspond to the epoch with the lowest val loss. The code is provided in `resnet-50.ipynb`.

The ResNet-50 results show given a strong enough architecture, the model can achieve high exact match accuracy. However, there is still a clear gap between training and validation performance, indicating that the model is overfitting the training data. Therefore, the following experiments focus on regularization methods and training strategies designed to improve generalization and reduce this performance gap.

2.3 ResNet-18

The previous ResNet-50 experiment achieved high training performance but showed a large gap between the training and test results. This suggested that the model had more capacity than necessary for the Multi-label MNIST dataset and was memorizing the training samples. We therefore selected the smaller ResNet-18 [2] architecture as a possible form of architectural regularization. Compared with ResNet-50, ResNet-18 contains fewer residual blocks and fewer parameters, while still retaining the main advantages of residual learning, including skip connections and stable gradient propagation.

The model begins with a 7×7 convolution, batch normalization, ReLU activation, and max pooling. It then uses four residual stages containing 2, 2, 2, and 2 basic residual blocks. The number of channels increases from 64 to 512 across the stages. Global average pooling is followed by a 256-unit fully connected layer and dropout before the final ten sigmoid outputs.

The optimizer was changed from Adam to SGD with momentum, using a learning rate of 0.1 and momentum of 0.9. The batch size was increased to 256, and the maximum number of epochs was set to 200. Early stopping monitored validation exact-match accuracy with a patience of 10, while ReduceLROnPlateau monitored validation loss with a patience of 5. This gives the learning-rate scheduler more opportunities to reduce the learning rate before training is terminated.

Table 11: Performance of the plain ResNet-18 model.

Split	Exact Match	Binary Accuracy	Precision	Recall
Train	0.9975	0.9997	0.9999	0.9996
Test	0.5504	0.9362	0.9442	0.9324

Training stopped at epoch 42, and the weights from epoch 32 were restored because this epoch achieved the highest validation exact-match accuracy. Although the model generalized better than the ResNet-50 baseline, the training exact-match accuracy remained substantially higher than the test accuracy. This confirms that overfitting is still present. Consequently,

the following experiments focus more strongly on regularization, learning-rate scheduling, and additional sources of supervision. The code is provided in **`resnet-18-plain.ipynb`**.

2.4 ResNet-18 + Regularization

Given the initial results, we applied numerous techniques of regularization to close the gap between train and val exact match accuracy. The following components were evaluated as potential methods for reducing overfitting.

2.4.1 Parameters Tuning and Learning Rate Scheduling

After observing overfitting in the plain ResNet-18 model, the first regularization experiment focused on reducing the capacity of the classification head and strengthening the training constraints. The convolutional backbone was kept unchanged so that the effect of the regularization changes could be studied more clearly.

The fully connected layer was reduced from 256 to 128 units. This limits the number of parameters in the classification head and reduces the opportunity for the model to memorize training-specific patterns. Dropout was also increased from 0.3 to 0.5, meaning that a larger proportion of the features is randomly removed during training. This encourages the network to rely on more distributed representations instead of a small number of strongly activated features.

Weight decay was enabled with a value of 5×10^{-3}. Weight decay penalizes large parameter values and provides an additional constraint on the model during optimization. The optimizer remained SGD with momentum, using a learning rate of 0.1 and momentum of 0.9.

The callback configuration was also adjusted. ReduceLROnPlateau continued to monitor validation loss, but its patience was increased from 5 to 10 epochs. Early stopping monitored validation exact-match accuracy and used a patience of 20 epochs. Keeping the early-stopping patience larger than the learning-rate-scheduler patience allows the optimizer to reduce the learning rate and continue exploring the local region of the solution before training is terminated. This is particularly important because validation loss and exact-match accuracy do not always improve at the same time.

Overall, this experiment combines architectural regularization, stochastic feature removal, weight regularization, and a more patient training schedule. The backbone and main optimization strategy were intentionally kept fixed so that later experiments could build on this configuration.

2.4.2 Histogram Prediction

The multi-hot label only indicates whether a digit appears in an image. It does not distinguish between one occurrence and multiple occurrences of the same digit. To provide the model with richer supervision, an auxiliary histogram prediction task was added.

For each image, a 10-dimensional histogram was constructed from `all_digit_labels`. The i-th element of the histogram represents the number of times digit i appears in the image. For example, the digit sequence

$$[6, 8, 8, 2, 8, 8]$$

produces the histogram

$$[0, 0, 1, 0, 0, 0, 1, 0, 4, 0].$$

The model uses a shared ResNet-18 feature extractor followed by two output heads. The original multi-label head remains unchanged and uses ten sigmoid outputs. The auxiliary histogram head consists of ten linear outputs, one for each digit count. During training, the model receives both targets:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{multi}}\mathcal{L}_{\text{BCE}} + \lambda_{\text{hist}}\mathcal{L}_{\text{hist}}$$

Binary cross-entropy is used for the multi-label output, while a regression loss is used for histogram prediction. In this experiment, the histogram loss was implemented using mean absolute error. The additional task forces the shared backbone to learn features that capture not only digit presence, but also repeated occurrences of the same digit.

Several loss-weight configurations were tested. The best performance was obtained with equal weights:

$$\lambda_{\text{multi}} = 0.5, \qquad \lambda_{\text{hist}} = 0.5.$$

This configuration gives the primary multi-label task and the auxiliary histogram task equal influence during optimization.

2.4.3 Contrastive Learning and Embedding Loss

This experiment was inspired by SimCLR [3], particularly its use of learned vector representations and similarity-based objectives. However, instead of defining strict positive and negative pairs, we assign a continuous similarity score to every pair of images. Strict pair construction could discard useful information because two images may be neither completely similar nor completely different, as defined by their histograms.

The model adds an independent embedding branch directly after global average pooling. A dense layer projects each feature vector into a 128-dimensional representation, which is then L2-normalized. The classification and histogram heads remain unchanged.

For each image, the digit-count histogram is normalized by the total number of digits. The cosine similarity between every pair of normalized histograms is used as the target similarity score. The model produces a corresponding similarity score by computing the dot product between normalized embeddings and mapping it from $[-1, 1]$ to $[0, 1]$.

The embedding loss is the mean absolute error between the predicted and target similarity matrices:

$$\mathcal{L}_{\text{embedding}} = \frac{1}{B^2}\sum_{i=1}^{B}\sum_{j=1}^{B}\left|S_{ij}^{\text{pred}} - S_{ij}^{\text{target}}\right|.$$

Experimenting with the weights shows that equal weights between the BCE loss and embedding loss give the best result. Therefore, combined with histogram loss, the final training objective combines equal weights between the multi-label classification loss, histogram loss, and embedding loss:

$$\mathcal{L} = 0.33\mathcal{L}_{\text{BCE}} + 0.33\mathcal{L}_{\text{histogram}} + 0.33\mathcal{L}_{\text{embedding}}.$$

This gives the model additional supervision about the relationship between samples while preserving the original multi-label classification task.

2.4.4 Drop Path

To reduce overfitting, stochastic depth, also called DropPath, was added to the ResNet-18 backbone. This technique was introduced by Huang et al. [4] and is particularly suitable for residual networks. During training, the residual branch of each block can be randomly skipped, while the shortcut branch is always preserved. During inference, all residual branches are active.

In the implementation, a custom `StochasticDepth` layer is inserted after the second batch-normalization layer of each residual block and before the residual branch is added to the shortcut. When the residual branch is retained, its output is scaled by the inverse of its keep probability. This preserves the expected magnitude of the activations. When stochastic depth is disabled, the layer behaves as an identity operation and the original ResNet-18 architecture is recovered.

The drop probability is increased linearly across the eight residual blocks. Therefore, shallow blocks receive little or no stochastic-depth regularization, while deeper blocks receive stronger regularization. The maximum drop probability is controlled by the `MAX_DROP_PATH` parameter. Several values were tested experimentally, and a maximum drop probability of 0.2 produced the best validation performance. This value was therefore used in the final configuration.

All other parts of the model remained unchanged, including the classifier dropout layer, optimizer, learning-rate scheduling, loss functions, output heads, and training pipeline. The DropPath experiment therefore isolates the effect of stochastic-depth regularization on the ResNet-18 backbone.

2.4.5 Data Generation

The dataset-generation procedure is discussed in detail in Section 1.3. The pipeline was used here to create additional training samples from the original MNIST training set. The generator reused only the spatial metadata from `train.pt`, specifically the `all_positions` field, and did not reuse the original images or labels.

For each generated sample, a position template was selected from the original training metadata. A digit class was then sampled independently and uniformly from the ten classes for every valid position. To avoid repeatedly selecting the same source examples too early, each digit class was stored in a shuffled queue. The queue was consumed sequentially and reshuffled only after all examples from that class had been used. Each selected MNIST digit was pasted into the corresponding 28×28 region of a blank 64×64 canvas. Only nonzero pixels overwrote the canvas, so black background pixels remained transparent.

The generated dataset was added to the original training set, while the validation and test sets were kept unchanged. Different amounts of generated data were evaluated. Adding 100,000 images to the original training set gave the best result. Increasing the amount of synthetic data beyond this point produced diminishing returns and, in some experiments, even reduced performance. This suggests that additional data was helpful up to a certain point, but excessive synthetic data may introduce a distribution shift or reduce the relative influence of the original samples.

Because the model had substantially more training data available, the capacity of the classification head was increased. The fully connected layer was changed from 128 to 512 units, and the dropout rate was reduced from 0.5 to 0.3. The larger dense layer allowed the model to learn richer representations from the expanded dataset, while the lower dropout rate avoided unnec-

essarily limiting this additional capacity. The separate 128-dimensional embedding branch was kept unchanged.

These experiments combined gives our best result:

Table 12: ResNet-18 with regularization.

Split	**Exact Match**	**Binary Accuracy**	**Precision**	**Recall**
Train	0.9760	–	0.9977	0.9968
Test	0.8545	0.9810	0.9821	0.9814

The code is provided in **`resnet-18-final.ipynb`**

3 Results

Table 13 summarizes the main experiments conducted in this project.

Table 13: Test-set performance of the evaluated models.

Model	Exact Match	Binary Accuracy	Precision	Recall
Baseline CNN	0.1838	0.8394	0.8701	0.8127
ResNet-50	0.3985	0.9045	0.9227	0.8911
Plain ResNet-18	0.5504	0.9362	0.9442	0.9324
ResNet-18 + Regularization	**0.8545**	**0.9810**	**0.9821**	**0.9814**

References

[1] Y. LeCun, C. Cortes **and** C. Burges, **?**MNIST handwritten digit database,**?** *ATT Labs [Online]*, **jourvol** 2, 2010.

[2] K. He, X. Zhang, S. Ren **and** J. Sun, *Deep Residual Learning for Image Recognition*, 2015. arXiv: 1512.03385 [cs.CV]. **url**: https://arxiv.org/abs/1512.03385

[3] T. Chen, S. Kornblith, M. Norouzi **and** G. Hinton, **?**A Simple Framework for Contrastive Learning of Visual Representations,**?** **in***Proceedings of the 37th International Conference on Machine Learning* **jourser** Proceedings of Machine Learning Research, **volume** 119, PMLR, 2020, **pages** 1597–1607. **url**: https://proceedings.mlr.press/v119/chen20j

[4] G. Huang, Y. Sun, Z. Liu, D. Sedra **and** K. Q. Weinberger, **?**Deep Networks with Stochastic Depth,**?** **in***European Conference on Computer Vision* Springer, 2016, **pages** 646–661.